

TEEM: Time-Centered Event Memory for Agents

Anonymous ACL submission

Abstract

Large language models (LLMs) have enabled increasingly capable agents, yet long-horizon conversational memory remains a major challenge. While existing approaches have made progress, they typically treat time as auxiliary metadata, limiting reasoning over evolving interaction histories. We propose **TEEM** (Time-Centered Event Memory), a graph-based memory architecture that organizes interaction history around explicit temporal structure. TEEM constructs a temporally grounded memory graph in which temporal segments organize interaction events and link them to supporting evidence and evolving states. At query time, TEEM retrieves and fuses information from complementary memory layers, enabling temporal localization, event-level reasoning, evidence-grounded answering, and reasoning over state changes. Experiments on LoCoMo and LongMemEval show that TEEM consistently improves temporal reasoning and cross-session conversational memory, achieving an overall score of 95.67 on LoCoMo and 87.60 on LongMemEval. We will release our code soon.

1 Introduction

Large language models (LLMs) have enabled increasingly capable agents (Xi et al., 2023a; Park et al., 2023a; Wang et al., 2023a). Yet long-term memory remains a major bottleneck in persistent, multi-session interactions (Maharana et al., 2024a; Wu et al., 2025a). To behave consistently over extended interactions, an agent must not only recall past interactions, but also determine when they occurred, how they relate to subsequent updates, and which specific utterances support the answer.

A common strategy is to externalize interaction history and retrieve relevant content at inference time (Lewis et al., 2021, 2020; Borgeaud et al., 2022; Ram et al., 2023; Ma et al., 2023). However, unlike static document collections, interaction

histories evolve over time, contain user-specific updates, and often require recovering specific dialogue evidence rather than semantically similar text. Many questions over such histories therefore require explicit temporal reasoning, such as identifying preference changes, determining the order of events, or recalling prior discussions (Maharana et al., 2024a; Wu et al., 2025b). Flat similarity-based retrieval is thus often inadequate, especially for multi-hop reasoning over temporally distributed interactions (Tang and Yang, 2024).

Recent work has introduced explicit memory architectures for LLM-based systems, including memory-centric pipelines, agentic memory systems, and graph-structured memories (Chhikara et al., 2025a; Xu et al., 2025a; Rasmussen et al., 2025a). These studies show that structured memory can substantially improve long-horizon interaction. However, most existing approaches focus primarily on memory storage, consolidation, and lifecycle management.

Even when temporal information is incorporated, it is often attached to stored facts, relations, or sessions as auxiliary metadata for filtering, ranking, or historical maintenance, rather than used as the primary organizing principle for retrieval. This limitation also applies to temporally aware graph memories such as Zep (Rasmussen et al., 2025a), which model evolving facts and relations over time but do not explicitly structure retrieval around temporally localized interaction units and their supporting evidence. Consequently, existing systems remain limited on questions that require explicit temporal localization, cross-session dependency tracking, and reasoning about changes over time, even in the presence of stronger long-context modeling (Dai et al., 2019; Beltagy et al., 2020; Zaheer et al., 2020; Xiong et al., 2024; Liu et al., 2024).

We argue that *time should be treated as a first-class organizing principle of interaction memory*. Under this view, the natural semantic units aligned

to the timeline are not isolated utterances, which are often too fine-grained, but interaction *events*, such as a preference update, a planning discussion, or a correction to a previous statement. At the same time, event abstraction alone is insufficient: accurate answering often requires access to the underlying utterance-level evidence, as well as representations of evolving states, such as changing preferences, commitments, or object locations.

Motivated by these observations, we propose **TEEM (Time-Centered Event Memory)**, a memory architecture for long-horizon interactive systems. TEEM organizes interaction history as a heterogeneous memory graph in which temporal segments provide explicit temporal structure, and events, evidence, contextual entities, and evolving states are anchored to that structure. On top of this representation, TEEM performs route-specialized retrieval across complementary memory layers, enabling the system to combine temporal localization, event-level abstraction, evidence grounding, and state-aware context according to the needs of the query. Our contributions are as follows:

- We propose **TEEM**, a **time-centered event memory architecture** for long-horizon interaction history, in which temporal structure serves as the primary organizing axis for retrieval.
- We introduce an **event–evidence–state representation** that supports coherent interaction abstraction, utterance-level grounding, and modeling of state changes across interactions.
- We develop a **route-specialized retrieval and fusion mechanism** that combines complementary memory routes to support question answering over long-horizon interaction history.

2 Related Work

2.1 Retrieval-Augmented Generation and External Memory

Early work on augmenting language models with external memory is closely connected to retrieval-augmented generation (RAG). RAG combines parametric language models with non-parametric memory by retrieving relevant documents from an external corpus to support generation (Lewis et al., 2020). This line of work established the importance of explicit external retrieval for overcoming the limits of purely parametric knowledge. How-

ever, classic RAG primarily focuses on retrieving static passages from document collections, rather than organizing long-horizon interaction history as structured memory for conversational question answering.

2.2 Memory Architectures for LLM Agents

Recent work has explored augmenting large language model (LLM) systems with external memory to support long-horizon interactions beyond the immediate context window. Early work such as Generative Agents stores natural-language experience traces, synthesizes higher-level reflections, and retrieves them dynamically to guide future behavior (Park et al., 2023b). MemGPT formulates long-term memory as a hierarchical context management problem, enabling models to move information across memory tiers to mitigate context window limitations (Packer et al., 2024). More recent systems introduce explicit memory pipelines for extracting, consolidating, storing, and retrieving information from extended interaction histories, as exemplified by Mem0, MemoryOS, and Ever-MemOS (Chhikara et al., 2025a; Li et al., 2025a; Hu et al., 2026). These studies highlight the importance of persistent memory infrastructure for scalable LLM agents. However, most existing systems primarily focus on memory lifecycle management, such as what to store and how to maintain it over time, rather than how interaction history should be structured to support retrieval for conversational question answering.

2.3 Temporal Memory and Time-Aware Retrieval

Another line of work investigates how temporal information can be incorporated into conversational memory. Representative efforts include time-sensitive memory management and temporal knowledge graph based memory (Rasmussen et al., 2025a). In particular, Zep organizes conversational data through a temporally aware knowledge graph that maintains evolving relations over time (Rasmussen et al., 2025a).

However, existing temporal memory systems remain largely knowledge-centric: memory is organized around entities, facts, or relations, while time is attached as metadata describing when knowledge becomes valid or changes. As a result, they are less suited for tasks that require localizing specific interaction events or recovering dialogue evidence from long interaction histories. In contrast, our approach

treats time as the primary organizing backbone of memory and constructs memory around temporally localized interaction events, with dialogue evidence and contextual states explicitly represented as retrieval primitives.

Additional lines of related work include memory systems and agent-memory benchmarks that study long-term personalization, retrieval quality, and memory operating-system design in LLM agents (Wang et al., 2025; Chhikara et al., 2025b; Rasmussen et al., 2025b; Wu et al., 2025c; Tian et al., 2025; Nan et al., 2025; Fang et al., 2026; Wang and Chen, 2025; Xia et al., 2024; Xu et al., 2025b; Sun and Zeng, 2025; Li et al., 2025b; Jiang et al., 2025; Ferrag et al., 2025; Yehudai et al., 2025; Yang et al., 2025a,b). Complementary work on long-context and memory-augmented modeling investigates recurrent and compressive mechanisms, retrieval-aware training, and context-length scaling effects (Graves et al., 2014, 2016; Miller et al., 2016; Bulatov et al., 2022, 2023; Press et al., 2022; Liu et al., 2025; Wang et al., 2023b; Chen et al., 2024b, 2025; Zhang et al., 2025; Xi et al., 2023b). We also note connections to continual learning, model editing, and memory theory perspectives that inform how persistent representations can be updated and consolidated over time (De Lange et al., 2022; Sinitin et al., 2020; Meng et al., 2023; Ke et al., 2023; Hu et al., 2022; McClelland et al., 1995; Frankland and Bontempi, 2005; Chen et al., 2024a; Zhong et al., 2024; Szelogowski, 2025; Alonso et al., 2024; Maharana et al., 2024b; Rasmussen et al., 2025c; Shinn et al., 2024).

3 Method

We propose **TEEM** (Time-Centered Event Memory), a memory architecture for question answering over long-horizon interaction histories. The core idea of TEEM is to organize memory around an explicit temporal backbone and to represent interaction history using retrieval-oriented memory units, including temporal segments, events, evidence, contextual elements, and evolving states. This design enables the system to localize relevant interactions in time, retrieve semantically coherent event-level memories, recover supporting utterances, and reason over temporally evolving information across sessions.

A key design principle of TEEM is that memory structure should align with retrieval requirements. In long-horizon dialogue, questions rarely target

a single isolated utterance. Instead, they often require identifying *when* something happened, recovering the interaction event in which it occurred, grounding the answer in specific utterances, and tracing how information about a person, object, preference, or plan changed over time. TEEM is designed explicitly for this setting through a time-centered event memory graph and a set of complementary retrieval routes defined over that graph.

3.1 Time-Centered Event Memory Graph

Let an interaction history be a sequence of utterances

$$U = \{u_1, \dots, u_T\}, \quad (1)$$

where each utterance is represented as

$$u_t = (x_t, r_t, \tau_t), \quad (2)$$

with text content x_t , speaker role r_t , and timestamp τ_t .

From U , we construct a heterogeneous memory graph

$$G = (V, E), \quad (3)$$

whose node set is defined as

$$V = V_{\text{seg}} \cup V_{\text{evt}} \cup V_{\text{evd}} \cup V_{\text{ctx}} \cup V_{\text{state}}, \quad (4)$$

where V_{seg} , V_{evt} , V_{evd} , V_{ctx} , and V_{state} denote temporal segment nodes, event nodes, evidence nodes, contextual nodes, and state nodes, respectively.

The graph is produced by an LLM-based memory structuring module that identifies temporal boundaries, groups utterances into coherent interaction events, extracts salient contextual elements, and tracks state changes over time. Instead of attaching timestamps as auxiliary metadata to otherwise flat memory items, TEEM makes temporal organization explicit in the graph itself. Each node type is introduced because it supports a distinct retrieval role.

Temporal segments. We first partition the utterance stream into an ordered set of temporal segments

$$\Sigma = \{\sigma_1, \dots, \sigma_M\}, \quad (5)$$

where each segment covers a contiguous temporal interval. Each utterance is assigned to exactly one segment through a segmentation function

$$g_{\text{seg}}(u_t) = \sigma_m. \quad (6)$$

Segment nodes provide coarse temporal localization and are connected by adjacency edges, forming

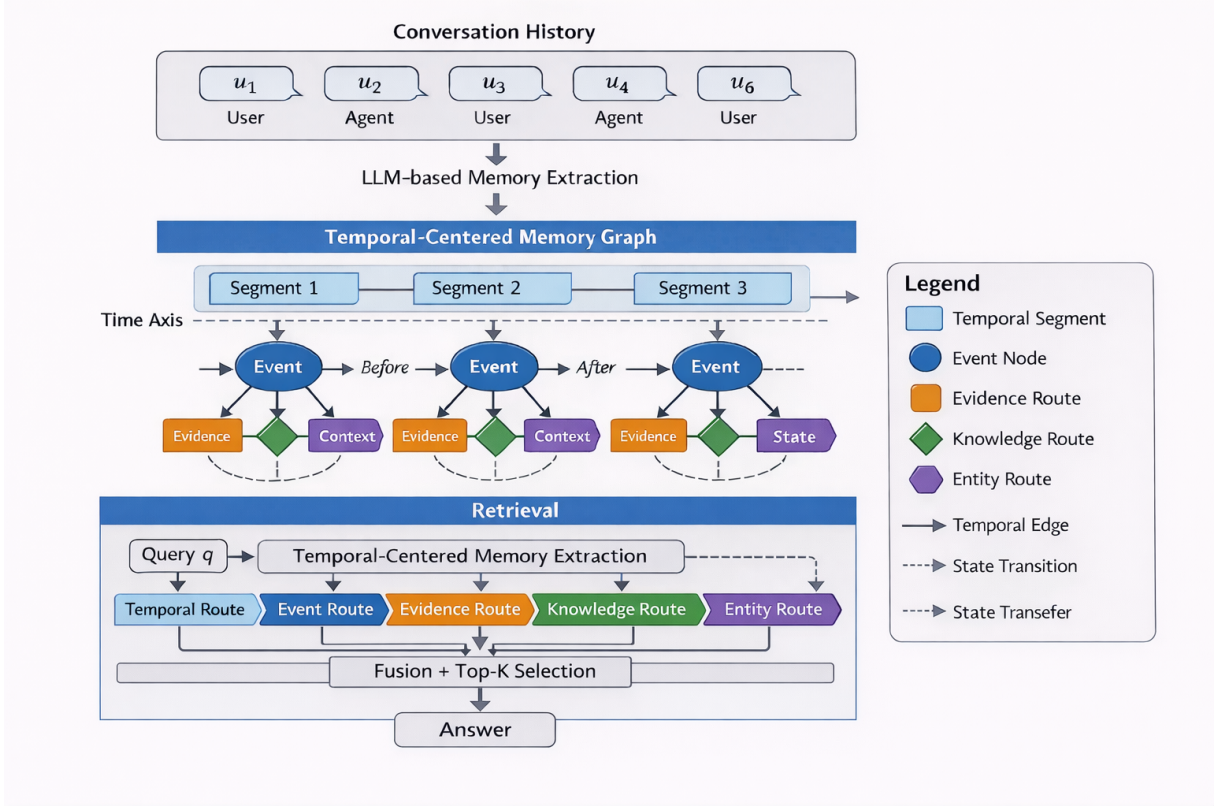


Figure 1: Overview of the proposed TEEM framework.

an explicit chronological backbone over the interaction history. Their role is to support retrieval over *time*: before searching for finer-grained memory items, TEEM can first identify the relevant temporal region.

Event nodes. Within each segment, the structuring module groups semantically coherent utterances into interaction events. Formally, an event node $e_i \in V_{\text{evt}}$ is defined as

$$e_i = \{u_t \mid z_t = i\}, \quad (7)$$

where z_t is the event assignment of utterance u_t . Each event is represented by

$$h_{\text{evt}}(e_i) = f_{\text{evt}}(\{u_t \mid z_t = i\}), \quad (8)$$

where $f_{\text{evt}}(\cdot)$ denotes an event encoder applied either to the grouped utterances or to an LLM-generated event summary. Event nodes serve as the primary semantic retrieval units in TEEM, since many dialogue phenomena of interest—such as planning, preference updates, clarifications, and corrections—span multiple turns and are more naturally represented as coherent events than as isolated utterances.

Evidence nodes. To preserve answer grounding, each utterance is also retained as an evidence node $d_t \in V_{\text{evd}}$ with representation

$$h_{\text{evd}}(d_t) = f_{\text{evd}}(x_t), \quad (9)$$

and linked to the event that it supports. Evidence nodes allow TEEM to recover exact supporting statements when fine-grained grounding is required, such as identifying a specific prior utterance, correction, or quoted preference. Thus, event nodes provide abstraction, while evidence nodes preserve traceability.

Contextual nodes. We further introduce contextual nodes for salient entities, objects, locations, or referents mentioned in the interaction history, and connect them to the events in which they appear. These nodes capture *which entity, object, location, or referent the interaction is about* and provide an access path when a query explicitly mentions such elements. In TEEM, contextual nodes are used to identify the referential target of a query, while temporally evolving information associated with that target is represented separately by state nodes.

State nodes. To explicitly model temporal evolution, we maintain state nodes of the form

$$s_k = (c, a, v, \tau), \quad (10)$$

where c is the associated contextual element, a is an attribute, v is its value, and τ is the update time. When an event changes a tracked attribute, a new state node is created and linked both to the triggering event and to the previous state of the same attribute, yielding a transition chain

$$s_{k-1} \rightarrow s_k. \quad (11)$$

State nodes explicitly represent temporally evolving information, such as changed preferences, revised plans, modified commitments, or status changes, rather than leaving such dynamics implicit in scattered utterances.

Importantly, state nodes are attached to contextual elements and to the events that trigger them. This makes state information directly searchable as part of the memory graph and allows TEEM to reason not only about what was mentioned, but also about what changed, when it changed, and which event caused the change.

Edge structure. The edge set includes segment adjacency, segment–event links, event–evidence links, event–context links, event–state links, temporal adjacency between nearby events, and state transition links:

$$E = E_{\text{seg}} \cup E_{\text{seg-evt}} \cup E_{\text{evt-evd}} \cup E_{\text{evt-ctx}} \cup E_{\text{evt-state}} \cup E_{\text{time}} \cup E_{\text{state}}. \quad (12)$$

Together, these nodes and edges define a memory graph centered on *time* and *event structure*: segments encode *when* relevant interactions occur, events encode *what* happened, evidence preserves traceability to original utterances, contextual nodes identify salient referential targets, and state nodes capture how interaction-relevant information evolves over time.

Because TEEM is designed for continual interaction, the memory graph is maintained incrementally rather than rebuilt from scratch. Given a new utterance u_{T+1} , the system first assigns it to a temporal segment,

$$\sigma^* = g_{\text{seg}}(u_{T+1}), \quad (13)$$

and then determines whether it should be attached to an existing event or start a new one,

$$e^* = g_{\text{evt}}(u_{T+1}, \sigma^*). \quad (14)$$

The utterance is always stored as a new evidence node, while contextual nodes and state chains are updated only when new entities or attribute changes are detected. This incremental update mechanism preserves the temporal and structural organization of memory without rebuilding the full graph from scratch.

3.2 Multi-Route Retrieval

Questions over long interaction histories require different forms of access to memory. Some queries ask about *when* something happened, some depend on matching a semantically relevant interaction event, and others require exact supporting utterances, contextual anchors, or temporally evolving information associated with a salient entity. TEEM therefore does not rely on a single retrieval space. Instead, it performs parallel retrieval over multiple complementary views induced by the memory graph.

Given a query q , TEEM retrieves candidates through five routes:

- **Temporal route**, which identifies relevant temporal segments and time-localized events;
- **Event route**, which retrieves semantically relevant event nodes;
- **Evidence route**, which retrieves utterance-level supporting evidence;
- **Context route**, which retrieves context nodes associated with salient elements explicitly mentioned in the query, such as people, objects, or locations;
- **Entity route**, which directly retrieves state nodes and linked events associated with a salient entity or referent.

Let

$$\mathcal{R} = \{\text{temp}, \text{event}, \text{evidence}, \text{context}, \text{entity}\} \quad (15)$$

denote the set of retrieval routes. Each route $r \in \mathcal{R}$ produces a ranked candidate list

$$R_r(q) = (m_{r,1}, m_{r,2}, \dots), \quad (16)$$

where each candidate m is a memory item in the graph, including event nodes, evidence nodes, context nodes, or state-linked memory.

These routes are specialized but complementary. The temporal route narrows the search space by

identifying likely time regions. The event route retrieves semantically coherent interaction units. The evidence route preserves access to exact supporting utterances. The context route grounds the query in the relevant contextual anchor, such as a person, location, object, or referent. The entity route then retrieves the temporally evolving state information associated with that target, together with events linked to those state updates.

Although we refer to the last route as an *entity route*, its retrieval target is state-linked memory. We use this name because user questions are often formulated around an entity or referent, while the underlying evidence needed for answering typically lies in the associated state trajectory and its linked events.

Route fusion. We combine route-specific rankings using weighted reciprocal rank fusion together with recency and graph-structural signals:

$$S(m | q) = \sum_{r \in \mathcal{R}} w_r \frac{1}{k + \text{rank}_r(m)} + \lambda_{\text{rec}} \text{Rec}(m) + \lambda_{\text{graph}} \text{Graph}(m | q), \quad (17)$$

where $r \in \mathcal{R} = \{\text{temp}, \text{event}, \text{evidence}, \text{context}, \text{entity}\}$ indexes the five retrieval routes, and w_r denotes the weight of route r . $\text{rank}_r(m)$ is the rank of memory item m under route r , $\text{Rec}(m)$ is a recency signal, and $\text{Graph}(m | q)$ is a graph-structural score that favors candidates supported by temporally adjacent, context-linked, or state-linked neighbors.

The recency term biases retrieval toward more recent information when competing candidates are otherwise similar, while still allowing older memories to be selected when strongly supported by route-specific evidence. The graph-structural term promotes coherent retrieval sets, for example when an event is supported by linked utterance evidence, when a contextual anchor connects to relevant events, or when an entity-linked state chain reveals informative transitions over time.

The final retrieved memory set is obtained by selecting the top- K candidates under the fused score:

$$\mathcal{M}_q = \text{TopK}_{m \in V} S(m | q). \quad (18)$$

Rather than forcing all queries into a single flat retrieval space, TEEM aligns retrieval with the temporal and structural organization of interaction memory. This design jointly supports temporal

localization, event-level abstraction, evidence recovery, contextual grounding, and reasoning over entity-associated state changes.

3.3 Answer Generation

Given a query q and the retrieved memory set $\mathcal{M}_q$, we serialize the selected memory items into a structured context

$$C(q, G) = \text{Serialize}(q, \mathcal{M}_q, G), \quad (19)$$

and provide it to the language model for answer generation:

$$a = \text{LLM}(q, C(q, G)). \quad (20)$$

The serialized context includes retrieved events together with linked evidence, contextual nodes, and relevant state updates. TEEM does not introduce a specialized generation module; its gains come from providing the language model with a temporally organized, structurally coherent, and evidence-grounded memory context.

4 Experiments

4.1 Experiment setting

We evaluate on LoCoMo and LongMemEval (Maharana et al., 2024a; Wu et al., 2025b), and compare against representative memory systems including Zep, Mem0, MemOS, MemoryOS, MemU, and EverMemOS (Rasmussen et al., 2025a; Chhikara et al., 2025a; Li et al., 2025a; Kang et al., 2025; Hu et al., 2026). To ensure fairness, we use a shared answer-generation backbone while keeping each baseline’s official memory configuration unchanged; for LongMemEval, we report baseline numbers from the official EverMemOS leaderboard.

Unless otherwise specified, experiments use GPT-4.1-mini, with GPT-4o-mini in selected comparisons. We fuse route-specific candidates with RRF ($k = 60$), then add recency and graph relevance ($\lambda_{\text{rec}} = 0.2$, $\lambda_g = 0.3$), and use Top-20 retrieved items for final context construction. Following prior work (Li et al., 2025a), each prediction is scored by GPT-4o-mini and averaged over three blind judgments. Detailed benchmark statistics, protocol design, and shared settings are provided in Appendix A.9 and Appendix A.10.

4.2 Main Results

The results in Tables 1 and 2 suggest that TEEM’s gains mainly come from better memory organization rather than from a stronger backbone or a

Backbone	Method	Avg. Input Tokens	Single Hop	Multi Hop	Temporal	Open Domain	Overall
GPT-4o-mini	MemoryOS	5.2k	62.43	56.50	37.18	40.28	54.70
	Mem0	1.0k	66.71	58.16	55.45	40.62	61.00
	MemU	4.0k	72.77	62.41	33.96	46.88	61.15
	MemOS	2.5k	81.45	69.15	72.27	60.42	75.87
	Zep	1.4k	88.11	71.99	74.45	66.67	81.06
	EverMemOS	2.5k	91.08	86.17	81.93	66.67	86.76
	TEEM	2.4k	95.69	93.55	92.50	89.58	92.83
GPT-4.1-mini	MemoryOS	5.5k	67.30	59.34	42.26	59.03	60.11
	Mem0	1.0k	68.97	61.70	58.26	50.00	64.20
	MemU	4.0k	74.91	72.34	43.61	54.17	66.67
	MemOS	2.5k	85.37	79.43	75.08	64.58	80.76
	Zep	1.4k	90.84	81.91	77.26	75.00	85.22
	EverMemOS	2.3k	96.67	91.84	89.72	76.04	93.05
	TEEM	1.7k	98.44	97.16	94.38	92.71	95.67

Table 1: Main results on LoCoMo.

Backbone	Method	Avg. Input Tokens	SS-User	SS-Asst	SS-Pref	Multi-S	Know. Upd	Temp. Reas	Overall
GPT-4.1-mini	MemU	0.5k	67.14	19.64	76.67	42.10	41.02	17.29	38.40
	Zep	1.6k	92.90	75.00	53.30	47.40	74.40	54.10	63.80
	Mem0	1.1k	82.86	26.78	90.00	63.15	66.67	72.18	66.40
	MemOS	1.4k	95.71	67.86	96.67	70.67	74.26	77.44	77.80
	EverMemOS	2.8k	97.14	85.71	93.33	73.68	89.74	77.44	83.00
	TEEM	2.3k	90.00	80.36	93.33	83.46	91.03	90.23	87.60

Table 2: Main results on LongMemEval (accuracy, %). All methods are evaluated under the same backbone (GPT-4.1-mini).

larger input context. On LoCoMo, TEEM achieves the best overall performance under both backbones, reaching **92.83** with GPT-4o-mini and **95.67** with GPT-4.1-mini. Under GPT-4o-mini, it outperforms EverMemOS by **6.07** points, with the largest gains on **Temporal** and **Open Domain**; under GPT-4.1-mini, TEEM still maintains clear advantages, especially on **Multi Hop**, **Temporal**, and **Open Domain**.

The key insight is that TEEM mainly improves *memory access* rather than *reasoning capacity*. A stronger backbone can reason better over retrieved evidence, but it does not by itself solve the problem of locating the right evidence in long interaction histories. By organizing memory around temporal structure and event-level units, TEEM makes retrieval better aligned with how conversational experience is naturally distributed across time and events.

A similar pattern appears on LongMemEval. TEEM achieves the best Overall score of **87.60**, and performs best on **Multi-S**, **Know. Upd**, and **Temp. Reas.**, with scores of **83.46**, **91.03**, and **90.23**, respectively. These results further suggest that TEEM is especially effective when the model must track updates, maintain consistency over long histories, and reason over temporally distributed

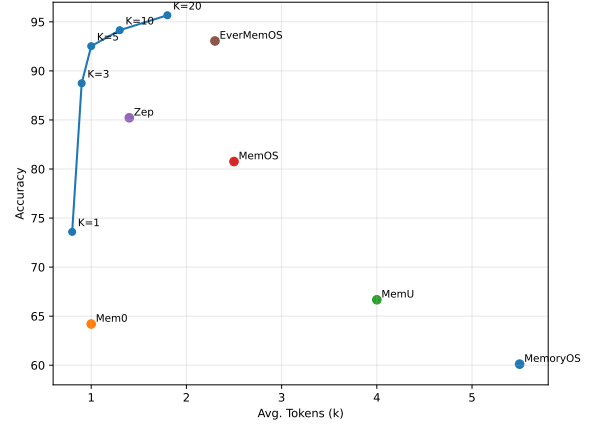


Figure 2: Performance and token efficiency under different retrieval depth K .

evidence.

4.3 Hyperparameter Analysis

We study the effect of **Top- K** on LoCoMo with GPT-4.1 MINI, where Top- K denotes the number of retrieved memory items used for QA. As shown in Figure 2, accuracy improves from 70.87 (Top-1) to 95.67 (Top-20) as context length increases from 0.8k to 1.8k tokens, exhibiting a clear accuracy–efficiency trade-off. The gains are front-loaded: increasing K from 1 to 3 yields the largest improve-

Top- K	Mean RT (ms)	P95 RT (ms)	Avg. Tok.
1	1398.2	3608.4	799.5
3	1490.0	3637.9	906.0
5	1526.1	3625.7	1009.9
10	1521.4	3691.2	1256.3
20	1466.8	3658.7	1750.4

Table 3: Efficiency under different Top- K retrieval settings. RT denotes retrieval time, and Tok. denotes average input (prompt) tokens.

	Accuracy	Gold Hit@20	Recall
TEEM (Full)	95.67	80.46	76.23
w/o Event Route	92.75	77.58	75.15
w/o Evidence Route	85.25	66.92	62.54
w/o Time Route	86.08	73.28	70.46

Table 4: Ablation study on retrieval routes. Each setting disables a specific retrieval route while keeping the underlying memory graph unchanged.

ment, while further increases bring diminishing returns. Compared with prior memory systems, Top- K variants achieve a stronger accuracy–efficiency frontier under similar or lower token budgets, with Top-5 already strong at low cost, Top-20 delivering the best overall performance at moderate cost (thus used as default), and Top-10 serving as a more efficient alternative.

4.4 Efficiency Analysis

Table 3 reports the efficiency of the system under different Top- K retrieval settings. As K increases from 1 to 20, average input (prompt) tokens rise from 799 to 1750, while retrieval latency remains stable for $K \geq 3$ (mean around 1.5s, P95 below 3.7s). This suggests that larger Top- K mainly increases generation context size rather than retrieval overhead, and the pipeline remains efficient at higher K .

4.5 Ablation

4.5.1 Different Routes

To analyze the role of different retrieval routes, we disable each route while keeping the underlying memory graph unchanged. As shown in Table 4, removing the **evidence route** leads to the largest drop, indicating that direct utterance-level retrieval is critical for grounding answers. Removing the **time route** also causes substantial degradation, showing that temporal signals are essential for organizing interaction history and guiding the model to the correct temporal context. In contrast, removing the **event route** results in only minor changes,

Setting	Accuracy
TEEM (Full)	95.67
w/o Event Nodes	73.73
w/o Evidence Nodes	72.27
w/o Temporal Nodes	70.91
w/o All Nodes(Vanilla)	0.70

Table 5: Node-level ablation. Retrieval is kept unchanged while different node types are masked from the final QA context.

suggesting that event-level retrieval mainly serves as a higher-level semantic abstraction. These results support our design that conversational memory should be structured around a temporal backbone, where time organizes events and evidence provides grounding.

4.5.2 Node-Level Ablation

To further examine how different memory components are used during answer generation, we mask specific node types from the final QA context while keeping retrieval unchanged. As shown in Table 5, removing any single node type causes a large performance drop, showing that the model depends on multiple complementary memory signals. Among them, masking **temporal nodes** leads to the largest degradation, confirming that temporal information is the core organizing backbone of the memory. Masking **evidence nodes** also severely hurts performance, indicating that utterance-level evidence is crucial for grounding answers, while removing **event nodes** results in a slightly smaller drop, suggesting that event nodes mainly provide structured abstractions. When all node types are removed, accuracy collapses to 0.70%, demonstrating that the task cannot be solved without structured memory inputs.

5 Conclusion

We presented **TEEM**, a time-centered memory architecture for agents. By structuring interaction history into a heterogeneous graph of events, evidence, and states, TEEM effectively captures both semantic abstractions and temporal dynamics. Experimental results show that TEEM achieves a state-of-the-art score of 92.83 on LoCoMo, significantly outperforming flat retrieval methods while maintaining high token efficiency. Our work demonstrates that treating time as a first-class organizing principle is crucial for building reliable, persistent conversational agents.

Limitations

This work has several limitations. First, TEEM is designed and evaluated primarily for text-based long-horizon conversational memory, and has not yet been extended to multimodal interaction scenarios involving modalities such as images or speech. Second, by organizing memory around temporal structure and event-level representations, TEEM introduces a somewhat more complex system design compared with simple flat retrieval approaches, which may increase implementation complexity. We view these limitations as reflecting the current scope of the work rather than fundamental constraints of the approach, and future work may explore multimodal memory modeling and more lightweight implementations.

References

- Nick Alonso, Tomás Figliolia, Anthony Ndirango, and Beren Millidge. 2024. [Toward conversational agents with context and time sensitive long-term memory](#). *Preprint*, arXiv:2406.00057.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. 2020. Longformer: The long-document transformer. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics.
- Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, and 1 others. 2022. Improving language models by retrieving from trillions of tokens. In *International Conference on Machine Learning*, volume 162, pages 2206–2240. PMLR.
- Atilim Gunes Bulatov, Valentin Khruikov, Leyla Mirvakhabova, Alexey Markov, Artem Babenko, and Ivan Oseledets. 2022. Recurrent memory transformer. In *Advances in Neural Information Processing Systems*, pages 20230–20243.
- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. 2023. [Scaling transformer to 1m tokens and beyond with rmt](#). *ArXiv*, abs/2304.11062.
- Guanzheng Chen, Xin Li, Zaiqiao Meng, Shangsong Liang, and Lidong Wang. 2024a. Clex: Continuous length extrapolation for large language models. In *International Conference on Learning Representations*.
- Guanzheng Chen, Xin Li, Michael Qizhe Shieh, and Lidong Bing. 2025. LongPO: Long context self-evolution of large language models through short-to-long preference optimization. In *The Thirteenth International Conference on Learning Representations*.

- Jun Chen, Dannong Xu, Junjie Fei, Chun-Mei Feng, and Mohamed Elhoseiny. 2024b. [Document haystacks: Vision-language reasoning over piles of 1000+ documents](#). *Preprint*, arXiv:2411.16740.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025a. [Mem0: Building production-ready ai agents with scalable long-term memory](#). *Preprint*, arXiv:2504.19413.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. 2025b. [Mem0: Building production-ready ai agents with scalable long-term memory](#). *Preprint*, arXiv:2504.19413.
- Zihang Dai, Zhilin Yang, Yiming Yang, Jaime Carbonell, Quoc V Le, and Ruslan Salakhutdinov. 2019. Transformer-XL: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988. Association for Computational Linguistics.
- Matthias De Lange, Rahaf Aljundi, Marc Masana, Sarah Parisot, Xu Jia, Aleš Leonardis, Greg Slabaugh, and Tinne Tuytelaars. 2022. A continual learning survey: Defying forgetting in classification tasks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(7):3366–3385.
- Jizhan Fang, Xinle Deng, Haoming Xu, Ziyang Jiang, Yuqi Tang, Ziwen Xu, Shumin Deng, Yunzhi Yao, Mengru Wang, Shuofei Qiao, Huajun Chen, and Ningyu Zhang. 2026. [Lightmem: Lightweight and efficient memory-augmented generation](#). *Preprint*, arXiv:2510.18866.
- Mohamed Amine Ferrag, Norbert Tihanyi, and Merouane Debbah. 2025. From llm reasoning to autonomous ai agents: A comprehensive review. *arXiv preprint arXiv:2504.19678*.
- Paul W Frankland and Bruno Bontempi. 2005. The organization of recent and remote memories. *Nature Reviews Neuroscience*, 6(2):119–130.
- Alex Graves, Greg Wayne, and Ivo Danihelka. 2014. Neural turing machines. *arXiv preprint arXiv:1410.5401*.
- Alex Graves, Greg Wayne, Malcolm Reynolds, Tim Harley, Ivo Keck, William O’Brien, Alistair Krizman, Stanislav Illarionov, Edward Grefenstette, Tiago Wuthrich, and 1 others. 2016. Hybrid computing using a neural network with dynamic external memory. *Nature*, 538(7626):471–476.
- Chuanrui Hu, Xingze Gao, Zuyi Zhou, Dannong Xu, Yi Bai, Xintong Li, Hui Zhang, Tong Li, Chong Zhang, Lidong Bing, and Yafeng Deng. 2026. [Evermemos: A self-organizing memory operating system for structured long-horizon reasoning](#). *Preprint*, arXiv:2601.02163.

716	Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan	Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao,	771
717	Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and	and Nan Duan. 2023. Query rewriting for retrieval-	772
718	Weizhu Chen. 2022. Lora: Low-rank adaptation of	augmented large language models . <i>Preprint</i> ,	773
719	large language models. In <i>International Conference</i>	arXiv:2305.14283.	774
720	on Learning Representations.		
721	Bowen Jiang, Yuan Yuan, Maohao Shen, Zhuoqun Hao,	Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov,	775
722	Zhangchen Xu, Zichen Chen, Ziyi Liu, Anvesh Rao	Mohit Bansal, Francesco Barbieri, and Yuwei Fang.	776
723	Vijjini, Jiashu He, Hanchao Yu, Radha Poovendran,	2024a. Evaluating very long-term conversational	777
724	Gregory Wornell, Lyle Ungar, Dan Roth, Sihao Chen,	memory of llm agents . <i>Preprint</i> , arXiv:2402.17753.	778
725	and Camillo Jose Taylor. 2025. Personamem-v2: To-		
726	wards personalized intelligence via learning implicit	Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov,	779
727	user personas and agentic memory. <i>arXiv preprint</i>	Mohit Bansal, Francesco Barbieri, and Yuwei Fang.	780
728	arXiv:2512.06688.	2024b. Evaluating very long-term conversational	781
		memory of llm agents . <i>Preprint</i> , arXiv:2402.17753.	782
729	Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting	James L McClelland, Bruce L McNaughton, and Ran-	783
730	Bai. 2025. Memory os of ai agent. <i>arXiv preprint</i>	dall C O'Reilly. 1995. Why there are complementary	784
731	arXiv:2506.06326.	learning systems in the hippocampus and neocortex:	785
		insights from the successes and failures of connec-	786
732	Zixuan Ke, Yijia Shao, Haowei Lin, Tatsuya Konishi,	tionist models of learning and memory. <i>Psychologi-</i>	787
733	Gyuhak Kim, and Bing Liu. 2023. Continual pre-	<i>cal review</i> , 102(3):419.	788
734	training of language models . In <i>International Con-</i>		
735	<i>ference on Learning Representations</i> .	Kevin Meng, Arnab Sharma, Alex Andonian, Yonatan	789
		Belinkov, and David Bau. 2023. Mass-editing mem-	790
736	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio	ory in a transformer. In <i>International Conference on</i>	791
737	Petroni, Vladimir Karpukhin, Naman Goyal, Yuxiang	<i>Learning Representations</i> .	792
738	Kukliansky, Wen-tau Yih Chen, Tim Rocktäschel,		
739	Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-	Alexander Miller, Adam Fisch, Jesse Dodge, Amir-	793
740	augmented generation for knowledge-intensive nlp	Hossein Karimi, Antoine Bordes, and Jason Weston.	794
741	tasks. In <i>Advances in Neural Information Processing</i>	2016. Key-value memory networks for directly read-	795
742	<i>Systems</i> , volume 33, pages 9459–9474.	ing documents. In <i>Proceedings of the 2016 Con-</i>	796
		<i>ference on Empirical Methods in Natural Language</i>	797
743	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio	<i>Processing</i> , pages 1400–1409. Association for Com-	798
744	Petroni, Vladimir Karpukhin, Naman Goyal, Hein-	putational Linguistics.	799
745	rich Küttler, Mike Lewis, Wen tau Yih, Tim Rock-		
746	täschel, Sebastian Riedel, and Douwe Kiela. 2021.	Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize	800
747	Retrieval-augmented generation for knowledge-	Chen. 2025. Nemori: Self-organizing agent	801
748	intensive nlp tasks . <i>Preprint</i> , arXiv:2005.11401.	memory inspired by cognitive science . <i>Preprint</i> ,	802
		arXiv:2508.03341.	803
749	Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding	Charles Packer, Vivian Woodside, Neal Dhir, and	804
750	Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Ji-	Douwe Kiela. 2024. Memgpt: Towards llms as op-	805
751	hao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao	erating systems. In <i>Advances in Neural Information</i>	806
752	Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Kehang Li,	<i>Processing Systems</i> .	807
753	Zhiqiang Yin, Qingchen Yu, Bo Tang, and 3 others.		
754	2025a. Memos: An operating system for memory-	Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai,	808
755	augmented generation (mag) in large language mod-	Meredith Ringel Morris, Percy Liang, and Michael S.	809
756	els . <i>Preprint</i> , arXiv:2505.22101.	Bernstein. 2023a. Generative agents: Interac-	810
		tive simulacra of human behavior . <i>Preprint</i> ,	811
757	Zhiyu Li, Shichao Song, Chenyang Xi, Hanyu Wang,	arXiv:2304.03442.	812
758	Chen Tang, Simin Niu, Ding Chen, Jiawei Yang,	Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai,	813
759	Chunyu Li, Qingchen Yu, and 1 others. 2025b.	Meredith Ringel Morris, Percy Liang, and Michael S.	814
760	Memos: A memory os for ai system. <i>arXiv preprint</i>	Bernstein. 2023b. Generative agents: Interac-	815
761	arXiv:2507.03724.	tive simulacra of human behavior . <i>Preprint</i> ,	816
		arXiv:2304.03442.	817
762	J Liu, D Zhu, Z Bai, Y He, H Liao, H Que, Z Wang,	Ofir Press, Noah A Smith, and Mike Lewis. 2022. Train	818
763	C Zhang, G Zhang, J Zhang, and Y Zhang. 2025.	short, test long: Attention with linear biases enables	819
764	A comprehensive survey on long context language	input length extrapolation. In <i>International Confer-</i>	820
765	modeling. <i>arXiv preprint arXiv:2503.17407</i> .	<i>ence on Learning Representations</i> .	821
766	Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paran-	Ori Ram, Eyal Shnarch, Jonathan Uziel, Lisa Haklay,	822
767	jape, Michele Bevilacqua, Fabio Petroni, and Percy	and Amir Globerson. 2023. In-context retrieval-	823
768	Liang. 2024. Lost in the middle: How language mod-	augmented language models. <i>Transactions of the</i>	824
769	els use long contexts. <i>Transactions of the Association</i>	<i>Association for Computational Linguistics</i> , 11:1316–	825
770	<i>for Computational Linguistics</i> , 12:157–173.	1331.	826

827	Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025a. Zep: a temporal knowledge graph architecture for agent memory. <i>arXiv preprint arXiv:2501.13956</i> .	881
828		882
829		883
830		884
831	Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025b. Zep: A temporal knowledge graph architecture for agent memory. <i>Preprint</i> , arXiv:2501.13956.	885
832		886
833		887
834		888
835	Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025c. Zep: A temporal knowledge graph architecture for agent memory. <i>Preprint</i> , arXiv:2501.13956.	889
836		890
837		891
838		892
839	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2024. Reflexion: Language agents with verbal reinforcement learning. In <i>Advances in Neural Information Processing Systems</i> , volume 36.	893
840		894
841		895
842		896
843		897
844	Anton Sinitsin, Vsevolod Plokhotnyuk, Dmitry Pyrkin, Sergei Popov, and Artem Babenko. 2020. Editable neural networks. In <i>International Conference on Learning Representations</i> .	898
845		899
846		900
847		901
848	Haoran Sun and Shaoning Zeng. 2025. Hierarchical memory for high-efficiency long-term reasoning in llm agents. <i>arXiv preprint arXiv:2507.22925</i> .	902
849		903
850		904
851	Daniel Szelogowski. 2025. Engram memory encoding and retrieval: A neurocomputational perspective. <i>Preprint</i> , arXiv:2506.01659.	905
852		906
853		907
854	Yixuan Tang and Yi Yang. 2024. Multihop-rag: Benchmarking retrieval-augmented generation for multihop queries. <i>arXiv preprint arXiv:2401.15391</i> .	908
855		909
856		910
857	Ao Tian, Yunfeng Lu, Xinxin Fan, Changhao Wang, Lanzhi Zhou, Yeyao Zhang, and Yanfang Liu. 2025. Rgm: Renormalization group-based memory evolution for language agent user profile. <i>Preprint</i> , arXiv:2510.16392.	911
858		912
859		913
860		914
861		915
862	Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2023a. Voyager: An open-ended embodied agent with large language models. <i>Preprint</i> , arXiv:2305.16291.	916
863		917
864		918
865		919
866		920
867	Piaohong Wang, Motong Tian, Jiaxian Li, Yuan Liang, Yuqing Wang, Qianben Chen, Tiannan Wang, Zhicong Lu, Jiawei Ma, Yuchen Eleanor Jiang, and Wangchunshu Zhou. 2025. O-mem: Omni memory system for personalized, long horizon, self-evolving agents. <i>Preprint</i> , arXiv:2511.13593.	921
868		922
869		923
870		924
871		925
872		926
873	Weizhi Wang, Li Dong, Hao Cheng, Xiaodong Liu, Xifeng Yan, Jianfeng Gao, and Furu Wei. 2023b. Longmem: Augmenting language models with long-term memory. In <i>Advances in Neural Information Processing Systems</i> , volume 36, pages 20292–20306.	927
874		928
875		929
876		930
877		931
878	Yu Wang and Xi Chen. 2025. Mirix: Multi-agent memory system for llm-based agents. <i>arXiv preprint arXiv:2507.07957</i> .	932
879		933
880		
	Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025a. Longmemeval: Benchmarking chat assistants on long-term interactive memory. <i>Preprint</i> , arXiv:2410.10813.	
	Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2025b. Longmemeval: Benchmarking chat assistants on long-term interactive memory. <i>Preprint</i> , arXiv:2410.10813.	
	Yaxiong Wu, Yongyue Zhang, Sheng Liang, and Yong Liu. 2025c. Sgm: Sentence graph memory for long-term conversational agents. <i>Preprint</i> , arXiv:2509.21212.	
	Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yi Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, and 1 others. 2023a. The rise and potential of large language model based agents: A survey. <i>arXiv preprint arXiv:2309.07864</i> .	
	Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Qin Liu, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, and 11 others. 2023b. The rise and potential of large language model based agents: A survey. <i>ArXiv</i> , abs/2309.07864.	
	Chun Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. 2024. Agentless: Demystifying llm-based software engineering agents. <i>ArXiv</i> , abs/2407.01489.	
	Wenhan Xiong, Jingyu Liu, Igor Molybog, Hejia Zhang, Prajjwal Bhargava, Rui Hou, Louis Martin, Rashi Rungta, Karthik Abinav Sankararaman, Barlas Oguz, and Madian Khabsa. 2024. Effective long-context scaling of foundation models. In <i>Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)</i> , pages 4643–4663.	
	Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025a. A-mem: Agentic memory for llm agents. <i>Preprint</i> , arXiv:2502.12110.	
	Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2025b. A-mem: Agentic memory for llm agents. <i>arXiv preprint arXiv:2502.12110</i> .	
	Zhongyu Yang, Jun Chen, Dannong Xu, Junjie Fei, Xiaoqian Shen, Liangbing Zhao, Chun-Mei Feng, and Mohamed Elhoseiny. 2025a. Wikiautogen: Towards multi-modal wikipedia-style article generation. <i>Preprint</i> , arXiv:2503.19065.	
	Zhongyu Yang, Yingfang Yuan, Xuanming Jiang, Baoyi An, and Wei Pang. 2025b. Inex: Hallucination mitigation via introspection and cross-modal multi-agent collaboration. <i>Preprint</i> , arXiv:2512.02981.	

Asaf Yehudai, Lilach Eden, Alan Li, Guy Uziel, Yilun Zhao, Roy Bar-Haim, Arman Cohan, and Michal Shmueli-Scheuer. 2025. Survey on evaluation of llm-based agents. *arXiv preprint arXiv:2503.16416*.

Manzil Zaheer, Guru Guruganesh, Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontañón, Philip Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. 2020. Big Bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, pages 17283–17296.

Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.

W Zhong, L Guo, Q Gao, and 1 others. 2024. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731.

A Additional Method and Experimental Details

This appendix provides additional technical details on the proposed TEEM framework, including the design objective of the memory graph, the role of each node and edge type, the incremental update mechanism, route-specialized retrieval, context serialization for answer generation, and the experimental protocol used in the main paper. We intentionally focus on clarifying the method and evaluation setup without introducing additional unreported quantitative results.

A.1 Notation and Design Objective

Let an interaction history be a sequence of utterances

$$U = \{u_1, \dots, u_T\}, \quad (21)$$

where each utterance is represented as

$$u_t = (x_t, r_t, \tau_t), \quad (22)$$

with utterance text x_t , speaker role r_t , and timestamp τ_t . From U , TEEM constructs a heterogeneous memory graph

$$G = (V, E), \quad (23)$$

where

$$V = V_{\text{seg}} \cup V_{\text{evt}} \cup V_{\text{evd}} \cup V_{\text{ctx}} \cup V_{\text{state}}. \quad (24)$$

Here, V_{seg} , V_{evt} , V_{evd} , V_{ctx} , and V_{state} denote temporal segment nodes, event nodes, evidence nodes, contextual nodes, and state nodes, respectively.

The design objective of TEEM is not merely to store historical content, but to organize memory in a form that is aligned with the retrieval demands of long-horizon conversational question answering. In this setting, an answer often depends on several complementary requirements:

- identifying *when* a relevant interaction took place;
- recovering *which event* or interaction episode the question refers to;
- locating the *supporting utterances* that ground the answer;
- disambiguating the *contextual target* of the question, such as a person, object, location, or referent;

- tracking *state changes over time*, such as preference updates, revised plans, or corrected information.

A flat memory store is often poorly aligned with this combination of needs. Retrieval over isolated utterances may preserve exact wording but fail to recover the correct interaction episode. Retrieval over only abstract summaries may improve semantic matching but lose grounding. Likewise, storing factual updates alone is insufficient for questions that depend on temporal ordering or interaction context. TEEM therefore treats time as the primary organizing axis and builds a memory graph whose structure directly supports temporal localization, event-level abstraction, evidence grounding, contextual anchoring, and reasoning over state evolution.

At query time, given a user question q , the system retrieves a set of memory items

$$\mathcal{M}_q = \text{TopK}_{m \in V} S(m \mid q), \quad (25)$$

where $S(m \mid q)$ is a fused score that combines route-specific relevance, recency, and graph-structural support, as described in the main paper. The central goal of TEEM is thus to make $\mathcal{M}_q$ both *relevant* and *structurally coherent*: rather than returning a flat collection of similar snippets, the retrieved set should expose a temporally localized and evidence-grounded interaction context that the language model can use reliably for answer generation.

A.2 Detailed Memory Graph Construction

TEEM constructs the memory graph using an LLM-based memory structuring module that operates over the utterance stream. This module identifies temporal boundaries, groups utterances into coherent interaction events, extracts salient contextual elements, and maintains state updates over time. The resulting representation is heterogeneous because each node type is intended to support a different retrieval function.

Temporal segmentation. The first step is to partition the utterance stream into an ordered sequence of temporal segments

$$\Sigma = \{\sigma_1, \dots, \sigma_M\}, \quad (26)$$

where each segment covers a contiguous temporal interval. Each utterance is assigned to exactly one segment by a segmentation function

$$g_{\text{seg}}(u_t) = \sigma_m. \quad (27)$$

The purpose of temporal segmentation is not simply to record timestamps, but to expose a retrieval layer that can localize relevant time regions at a coarser level than individual utterances. In long interaction histories, many questions first require narrowing the search to a relevant time span before finer-grained retrieval becomes effective.

Segment nodes are therefore connected by temporal adjacency edges, forming an explicit chronological backbone of the memory graph. This makes temporal structure part of the graph itself rather than leaving it as metadata attached to otherwise flat memory items.

Event induction. Within each temporal segment, TEEM groups semantically coherent utterances into interaction events. Formally, an event node $e_i \in V_{\text{evt}}$ is defined as

$$e_i = \{u_t \mid z_t = i\}, \quad (28)$$

where z_t is the event assignment of utterance u_t . An event representation is then computed as

$$h_{\text{evt}}(e_i) = f_{\text{evt}}(\{u_t \mid z_t = i\}), \quad (29)$$

where $f_{\text{evt}}(\cdot)$ denotes an event encoder applied to the grouped utterances or to a compact event-level summary derived from them.

The motivation for event nodes is that many conversational phenomena span multiple turns. Examples include making a plan, clarifying an earlier statement, correcting misinformation, updating a preference, or discussing an external event. Treating each utterance independently may fragment such phenomena, while event-level abstraction provides a more coherent retrieval unit for downstream reasoning.

Evidence preservation. Although event abstraction is useful, conversational question answering often still requires access to exact supporting utterances. TEEM therefore preserves every utterance as an evidence node $d_t \in V_{\text{evd}}$ with representation

$$h_{\text{evd}}(d_t) = f_{\text{evd}}(x_t). \quad (30)$$

Each evidence node is linked to the event in which the utterance appears. This design ensures that semantic abstraction does not come at the cost of answer grounding. Event nodes support semantic compression and coherence, while evidence nodes preserve traceability to the original dialogue.

Contextual anchoring. TEEM also introduces contextual nodes for salient entities, objects, locations, and referents mentioned in the interaction history. These nodes capture what the interaction is about and provide an access path when the query explicitly mentions such targets. A contextual node may correspond, for example, to a person under discussion, a place that appears in planning dialogue, an object that changes location, or a referential target that recurs across multiple sessions.

Contextual nodes are linked to the events in which they appear. Their role is not to replace state tracking, but to provide a stable referential anchor that allows the system to identify the correct target of a question before reasoning over the events and state updates associated with that target.

State tracking. To model temporally evolving information, TEEM maintains state nodes of the form

$$s_k = (c, a, v, \tau), \quad (31)$$

where c is the associated contextual element, a is an attribute, v is its value, and τ is the update time. Whenever an interaction event introduces or modifies a tracked attribute, a new state node is created and linked to the event that triggered the update. State nodes corresponding to successive updates of the same attribute are connected into transition chains,

$$s_{k-1} \rightarrow s_k. \quad (32)$$

This representation is particularly important for long-horizon interaction histories because user-related information often changes over time. A system must not only know the current value of a preference or plan, but also determine whether a question refers to an earlier value, a later correction, or the transition itself. State nodes make such temporal evolution explicit and directly searchable.

Edge structure. The edge set of the graph includes multiple relation types:

$$E = E_{\text{seg}} \cup E_{\text{seg-evt}} \cup E_{\text{evt-evd}} \cup E_{\text{evt-ctx}} \cup E_{\text{evt-state}} \cup E_{\text{time}} \cup E_{\text{state}}. \quad (33)$$

These relations encode chronological adjacency, containment across memory levels, and temporal evolution. Taken together, the graph provides a structured view of interaction history in which segments represent *when* something happened, events represent *what* happened, evidence nodes record

how it was said, contextual nodes identify *what or whom it concerns*, and state nodes represent *what changed over time*.

A.3 Node and Edge Semantics

Table 6 summarizes the semantics of the node types used in TEEM. Table 7 summarizes the main edge types and their functional role during retrieval and answer construction.

The distinction between contextual nodes and state nodes is especially important. A contextual node answers the question of *what or whom the interaction concerns*, while a state node answers the question of *what changed about that target*. This separation helps TEEM avoid collapsing referential identity and temporal evolution into a single memory object. In practice, many conversational questions require both: the model must first identify the correct referential target and then recover the time-sensitive information associated with it.

A.4 Incremental Graph Maintenance

TEEM is designed for continual interaction rather than one-shot offline indexing. Rebuilding the entire memory graph from scratch after every new utterance would be unnecessarily expensive and would fail to reflect the locality of most updates. Instead, TEEM maintains the graph incrementally.

Given a new utterance u_{T+1} , the update proceeds in several stages.

Step 1: temporal assignment. The system first determines the temporal segment to which the new utterance should belong:

$$\sigma^* = g_{\text{seg}}(u_{T+1}). \quad (34)$$

If the utterance falls into an existing temporal interval, it is attached to the corresponding segment. Otherwise, a new segment node is created and connected to neighboring segments along the chronological backbone.

Step 2: event attachment or creation. The system next decides whether the utterance continues an existing interaction event in the assigned segment or initiates a new event:

$$e^* = g_{\text{evt}}(u_{T+1}, \sigma^*). \quad (35)$$

This decision depends on whether the new utterance is semantically coherent with the current local interaction episode. If so, the utterance is attached to that event; otherwise, a new event node is created.

Node Type	Stored Content	Primary Role in Retrieval and Answering
Temporal segment node	A contiguous temporal region of the interaction history	Supports coarse temporal localization by identifying when relevant interactions are likely to have occurred.
Event node	A semantically coherent interaction episode composed of multiple utterances	Serves as the main semantic retrieval unit for plans, updates, clarifications, corrections, and other multi-turn interaction phenomena.
Evidence node	An individual utterance from the original dialogue	Preserves fine-grained grounding and enables recovery of exact supporting statements.
Contextual node	A salient person, object, location, or referential target mentioned in the interaction history	Anchors retrieval around the target explicitly referenced in the query and links related events across time.
State node	A temporally indexed attribute-value update associated with a contextual target	Models evolving information over time, such as changed preferences, revised plans, or corrected facts.

Table 6: Summary of node types in TEEM and their roles.

Edge Type	Meaning	Functional Benefit
Segment adjacency	Connects neighboring temporal segments in chronological order	Preserves the explicit timeline and supports temporal navigation across the interaction history.
Segment–event link	Connects an event to the temporal segment in which it occurs	Grounds each event in the temporal backbone and supports time-conditioned retrieval.
Event–evidence link	Connects an event to its constituent utterances	Allows event-level abstraction to be expanded into exact supporting evidence.
Event–context link	Connects an event to salient contextual targets mentioned in it	Supports referential access to events involving a queried person, object, or location.
Event–state link	Connects an event to the state update it triggers	Makes state changes explainable in terms of the interaction episode that caused them.
Temporal event adjacency	Connects nearby events across the timeline	Supports retrieval of temporally related interaction episodes beyond exact segment boundaries.
State transition link	Connects successive updates of the same attribute	Makes temporal evolution explicit and enables reasoning over changes rather than isolated facts.

Table 7: Summary of edge types in TEEM and their functional role.

Step 3: evidence insertion. Regardless of whether an existing event is reused or a new event is created, the utterance is always preserved as a new evidence node. This guarantees that the original dialogue remains recoverable even when higher-level event structure evolves over time.

Step 4: contextual update. The system then identifies whether the utterance introduces previously unseen salient targets or refers to existing ones. If a new contextual target is detected, a new contextual node is created. Otherwise, the utterance is linked to the relevant existing contextual node through its associated event.

Step 5: state update. If the utterance expresses an attribute-value update for a tracked contextual target, the system creates a new state node and links it both to the triggering event and to the previous state in the same attribute trajectory. In this way, the state chain grows locally and does not require reconstructing the full history.

Step 6: local structural consistency. Finally, local edges involving the updated event, contextual target, and state chain are refreshed so that newly added content is accessible through the same retrieval routes as older memory.

This incremental update process is important for two reasons. First, it aligns with the continual nature of interactive systems, where memory must evolve online. Second, it preserves the structural organization of TEEM without turning memory maintenance into a full re-indexing problem after every interaction.

A.5 Route-Specialized Retrieval

Questions over long interaction histories are heterogeneous, and TEEM addresses this by retrieving through multiple complementary routes. Let

$$\mathcal{R} = \{\text{temp}, \text{event}, \text{evidence}, \text{context}, \text{entity}\} \quad (36)$$

denote the set of routes. Each route $r \in \mathcal{R}$ produces a ranked list

$$R_r(q) = (m_{r,1}, m_{r,2}, \dots). \quad (37)$$

A generic view of route-specific matching is

$$s_r(q, m) = \text{Sim}(\phi_r(q), h_r(m)), \quad (38)$$

where $\phi_r(q)$ denotes a route-specific query representation, $h_r(m)$ denotes a route-specific memory representation, and $\text{Sim}(\cdot, \cdot)$ is the corresponding relevance function. TEEM does not require all routes to use the same retrieval representation, because each route targets a different aspect of the memory graph.

Temporal route. The temporal route aims to answer the question: *when should the system look?* It retrieves temporal segments and time-localized events that are likely to contain the answer. This route is especially useful for questions involving order, recency, change over time, or references to earlier and later interactions.

Event route. The event route aims to answer the question: *which interaction episode is semantically relevant?* It retrieves event nodes representing coherent multi-turn interaction units. This route is effective when a query refers to an entire episode, such as a planning discussion, a clarification exchange, or a preference update that unfolds over several utterances.

Evidence route. The evidence route aims to answer the question: *which original utterances provide direct support?* It retrieves utterance-level evidence nodes. This route is essential for grounding answers in specific dialogue content and is especially important when the correct answer depends on exact wording, explicit corrections, or fine-grained distinctions that might be blurred by event abstraction alone.

Context route. The context route aims to answer the question: *what is the query about?* It retrieves contextual nodes associated with salient targets explicitly mentioned or implied by the query, such as people, places, objects, or recurring referents. This route helps disambiguate which part of the interaction history should be examined.

Entity route. The entity route aims to answer the question: *what time-evolving information is associated with the target?* Although the route is query-facing and often triggered by an entity mention, its retrieval target is state-linked memory: state nodes and linked events associated with the contextual target. This route is particularly useful for questions about updates, changes, consistency

over time, or the current versus earlier value of some attribute.

Table 8 summarizes these retrieval routes.

The routes are complementary rather than redundant. Temporal retrieval localizes the search in time, event retrieval provides semantically coherent episodes, evidence retrieval preserves grounding, context retrieval identifies the referential anchor, and entity-route retrieval exposes state trajectories associated with that anchor. This decomposition is one of the central design choices of TEEM.

A.6 Fusion and Final Candidate Selection

The final retrieval score combines route-specific rankings with recency and graph-structural support:

$$S(m | q) = \sum_{r \in \mathcal{R}} w_r \frac{1}{k + \text{rank}_r(m)} + \lambda_{\text{rec}} \text{Rec}(m) + \lambda_{\text{graph}} \text{Graph}(m | q). \quad (39)$$

Here, w_r is the weight of route r , $\text{rank}_r(m)$ is the rank of candidate m under route r , $\text{Rec}(m)$ is a recency signal, and $\text{Graph}(m | q)$ is a graph-based structural support signal. The RRF parameter is set to 60 in the reported experiments, and the final ranking also incorporates $\lambda_{\text{rec}} = 0.2$ and $\lambda_g = 0.3$.

The use of weighted reciprocal rank fusion is important because different queries may rely on different routes to different extents. A query about a changed preference may receive strong evidence from the entity route and the temporal route, whereas a query requiring exact quotation may depend more strongly on the evidence route. RRF provides a robust mechanism for combining these heterogeneous rankings without forcing all routes into a single score space from the outset.

The recency term plays a secondary but useful role. In conversational settings, more recent information is often more relevant, especially when multiple candidate memories express competing states. However, recency alone is not sufficient: older information may still be correct if strongly supported by route-specific evidence. TEEM therefore uses recency as a bias rather than as the primary criterion.

The graph score favors candidates that are structurally supported by related memories in the graph. For example, an event may receive additional support if it is linked to retrieved evidence nodes, if it is connected to the contextual target identified by the query, or if it lies on a state trajectory that

Route	Primary Retrieval Target	Main Question It Helps Resolve	Typical Use Case
Temporal	Temporal segments and time-localized events	When did the relevant interaction occur?	Ordering, recency, prior versus later interactions, update tracking
Event	Event nodes	Which interaction episode is semantically relevant?	Planning discussions, clarifications, corrections, multi-turn episodes
Evidence	Evidence nodes	Which utterances directly support the answer?	Exact grounding, quoted statements, fine-grained distinctions
Context	Contextual nodes	What person, object, location, or referent is the query about?	Referential disambiguation and target identification
Entity	State nodes and linked events	What changed over time for the queried target?	Preference changes, revised plans, corrected facts, state evolution

Table 8: Summary of TEEM’s retrieval routes.

clarifies the relevant temporal change. This structural term encourages the final retrieved set to be internally coherent rather than merely individually relevant.

The final retrieved memory set is

$$\mathcal{M}_q = \text{TopK}_{m \in V} S(m \mid q), \quad (40)$$

where, unless otherwise specified, $K = 20$ in the main experiments.

A.7 Context Serialization for Answer Generation

After retrieval, the selected memory items are serialized into a structured context

$$C(q, G) = \text{Serialize}(q, \mathcal{M}_q, G), \quad (41)$$

which is then provided to the language model:

$$a = \text{LLM}(q, C(q, G)). \quad (42)$$

An important design choice is that TEEM does not pass the retrieved memory as a flat list of unrelated snippets. Instead, the serialized context preserves the organization induced by the memory graph as much as possible. In particular, event-level items are paired with their linked evidence when grounding is needed, contextual targets are included when they disambiguate the query, and state updates are surfaced when temporal evolution is important for answering.

A useful way to view the serialization step is as converting a heterogeneous graph neighborhood into a compact textual scaffold for the language model. In practice, this scaffold may include the following kinds of information:

- the retrieved event or events that most likely contain the answer;

- the supporting utterances attached to those events;
- the contextual targets that identify the subject of the question;
- the relevant state updates and their temporal ordering.

This organization is especially useful in long-horizon conversational question answering because the language model does not need to reconstruct all latent structure from a flat set of passages. Instead, the retrieved context already presents a temporally organized and semantically coherent view of the relevant memory neighborhood.

Notably, TEEM does not introduce a specialized generation module. The generation backbone is unchanged; the gains come from improving the quality and structure of the memory context presented to the model.

A.8 How Memory Layers Align with Question Types

The design of TEEM is motivated by the observation that different question categories place pressure on different parts of the memory system.

Single-hop questions. Single-hop questions are often answerable from a single interaction event or a small number of utterances. For such cases, the event route and evidence route are usually sufficient: the event route recovers the relevant episode, while the evidence route grounds the final answer in exact dialogue content.

Multi-hop questions. Multi-hop questions often require combining information across multiple interaction episodes, potentially separated in time. Here, event-level retrieval is important because

it recovers semantically coherent episodes rather than isolated lines, while the temporal backbone and graph connectivity help bridge related events across sessions or time segments.

Temporal questions. Temporal questions directly benefit from treating time as a first-class organizing principle. Such questions may require determining what happened first, whether a later interaction overrode an earlier one, or which value held at a particular point in time. Temporal segments and state transition chains are particularly useful in this setting.

Open-domain conversational questions. Open-domain questions in long-horizon dialogue often involve identifying the correct interaction context before reasoning further. Even when the answer is not a direct quote, the system still needs to recover the correct event, relevant evidence, and the appropriate context anchor. TEEM’s multi-route design helps recover this interaction context more reliably than a single flat retrieval space.

This alignment between memory layers and question types is one of the key reasons why a heterogeneous, time-centered memory can be more effective than either flat utterance retrieval or purely fact-centric memory organization.

A.9 Experimental Settings

Benchmarks. We evaluate TEEM on LoCoMo and LongMemEval (Maharana et al., 2024a; Wu et al., 2025b). LoCoMo is a benchmark of 10 very long multi-session conversations, with 1,986 question-answer pairs in the released evaluation set. In our evaluation, we report four question categories: 841 single-hop, 282 multi-hop, 321 temporal, and 96 open-domain questions. LongMemEval contains 500 questions and evaluates long-context conversational memory in settings that require tracking updates, maintaining consistency across long histories, and abstaining appropriately when the answer is unsupported.

These two benchmarks are complementary. LoCoMo highlights structured reasoning over conversational history with interpretable subcategories, while LongMemEval places additional emphasis on full-history understanding and robustness in extended interaction settings.

Baselines. We compare TEEM against representative memory systems, including Zep, Mem0, MemOS, MemoryOS, MemU, and EverMemOS

(Rasmussen et al., 2025a; Chhikara et al., 2025a; Li et al., 2025a; Kang et al., 2025; Hu et al., 2026). These baselines cover a range of memory designs, including memory pipelines, operating-system-style memory management, and temporally aware graph-based approaches.

Fairness considerations. For fair comparison, we use the same answer-generation backbone across methods while leaving each baseline’s official memory configuration unchanged. This setup isolates the effect of memory organization and retrieval quality as much as possible, rather than attributing performance differences to different generators.

Shared implementation settings. Unless otherwise specified, the main experiments use GPT-4.1-mini as the backbone language model, with GPT-4o-mini used in selected comparison settings. Candidates from the five retrieval routes are fused using RRF with parameter 60. In the final ranking, we additionally incorporate recency and graph-based relevance signals with weights $\lambda_{\text{rec}} = 0.2$ and $\lambda_g = 0.3$. The default retrieval depth is Top-20, meaning that the final answer context is constructed from the top 20 fused candidates.

The use of shared hyperparameters across the reported benchmark settings is important because it reduces the chance that gains are caused by aggressive task-specific tuning. In other words, TEEM is evaluated primarily as a memory design rather than as a heavily hand-tuned benchmark-specific pipeline.

A.10 Evaluation Protocol

Following prior work on memory-augmented evaluation, we adopt an LLM-as-a-judge protocol. Each prediction is scored by GPT-4o-mini, and the final score is computed as the average of three blind judgments. This evaluation strategy is intended to reduce variance from any single judgment and to provide a consistent assessment criterion across methods.

The use of blind repeated judgments is particularly useful in long-horizon conversational QA because model outputs may vary in wording while still being semantically correct. A strict string-match criterion would be overly brittle in this setting, especially for open-domain or paraphrastic answers. Averaging multiple blind judgments therefore provides a more robust estimate of answer quality.

A.11 Ablation Protocol

We conduct two forms of ablation in the main paper: route-level ablation and node-level ablation.

Route-level ablation. In route-level ablation, we disable one retrieval route at a time while keeping the underlying memory graph unchanged. This design isolates the contribution of a specific retrieval path to final retrieval quality and downstream QA performance. For example, removing the evidence route tests how much the system relies on direct utterance-level grounding, while removing the temporal route tests how much performance depends on explicit temporal localization.

An important advantage of this protocol is that it preserves the underlying memory representation. The graph still contains the same nodes and edges; only one access path is removed. This means the ablation measures the importance of the route itself rather than conflating retrieval-route removal with changes in the stored memory.

Node-level ablation. In node-level ablation, retrieval is kept unchanged, but selected node types are masked from the final QA context before answer generation. This tests a different question: once candidate memories have been retrieved, which kinds of information does the language model actually depend on when forming an answer?

Masking event nodes tests the contribution of event-level abstraction. Masking evidence nodes tests the importance of direct utterance-level grounding. Masking temporal nodes tests the role of explicit temporal structure. Finally, masking all node types collapses the structured memory signal and provides a strong lower-bound sanity check on how much the system depends on the memory representation.

Together, the two ablation protocols evaluate complementary aspects of the method: route-level ablation probes *how the system accesses memory*, while node-level ablation probes *what information the model uses after retrieval*.

A.12 Efficiency Interpretation

The efficiency results in the main paper show that increasing Top- K mainly increases the amount of context passed to the generator, while retrieval latency remains comparatively stable once K is no longer extremely small. This pattern is consistent with the design of TEEM.

The main computational work of retrieval lies in querying the route-specific memory views and fusing the resulting candidate lists. Increasing the final retained Top- K does not fundamentally change the cost of building the candidate pool; instead, it mostly affects how many items are serialized into the final answer context. As a result, increasing Top- K produces a clearer effect on average context length than on retrieval latency itself.

This observation is useful in practice because it suggests that accuracy-efficiency trade-offs in TEEM can be tuned largely through the retrieval depth without requiring a different memory structure. Smaller Top- K values yield shorter contexts and lower generation cost, while larger values preserve more evidence and improve accuracy, especially for questions that depend on multiple supporting memories.

A.13 Why the Appendix Focuses on Methodological Clarification

This appendix intentionally focuses on methodological clarification rather than adding new quantitative claims. The reason is that a strong appendix should improve reproducibility and make the design logic of the method easier to inspect, without introducing unsupported results that are not discussed in the main paper.

Accordingly, the appendix expands on the following aspects:

- how TEEM organizes memory into temporal, event, evidence, context, and state layers;
- how the graph is maintained incrementally over continual interaction;
- why multiple retrieval routes are necessary for heterogeneous conversational questions;
- how retrieved memories are fused and serialized into an answer-ready context;
- how the experimental and ablation protocols are defined.

We believe this level of detail better communicates the technical contribution of TEEM and makes the system design easier to evaluate in a fair and transparent way.

B Evaluation Prompts

This section presents the prompts used for answer generation and evaluation.

B.1 QA Prompt

You are an intelligent memory assistant tasked with retrieving accurate information from episodic memories.

CONTEXT:

You have access to episodic memories from conversations between two speakers. These memories contain timestamped information that may be relevant to answering the question.

INSTRUCTIONS:

Your goal is to synthesize information from all relevant memories to provide a comprehensive and accurate answer. You MUST follow a structured Chain-of-Thought process to ensure no details are missed. Actively look for connections between people, places, and events to build a complete picture. Synthesize information from different memories to answer the user’s question. It is CRITICAL that you move beyond simple fact extraction and perform logical inference. When the evidence strongly suggests a connection, you must state that connection. Do not dismiss reasonable inferences as “speculation.” Your task is to provide the most complete answer supported by the available evidence.

CRITICAL REQUIREMENTS:

1. NEVER omit specific names – use “Amy’s colleague Rob” not “a colleague” 2. ALWAYS include exact numbers, amounts, prices, percentages, dates, times 3. PRESERVE frequencies exactly – “every Tuesday and Thursday” not “twice a week” 4. MAINTAIN all proper nouns and entities as they appear

RESPONSE FORMAT (You MUST follow this structure):

STEP 1: RELEVANT MEMORIES EXTRACTION

[List each memory that relates to the question, with its timestamp]

- Memory 1: [timestamp] - [content] - Memory 2: [timestamp] - [content] ...

STEP 2: KEY INFORMATION IDENTIFICATION

[Extract ALL specific details from the memories]

- Names mentioned: [list all person names, place names, company names] - Numbers/Quantities: [list all amounts, prices, percentages] - Dates/Times: [list all temporal information] - Frequencies: [list any recurring patterns] - Other entities: [list brands, products, etc.]

STEP 3: CROSS-MEMORY LINKING

[Identify entities that appear in multiple memories and link related information. Make reasonable inferences when entities are strongly connected.]

- Shared entities: [list people, places, events mentioned across different memories] - Connections found: [e.g., “Memory 1 mentions A moved from hometown → Memory 2 mentions A’s hometown is LA → Therefore A moved from LA”] - Inferred facts: [list any facts that require combining information from multiple memories]

STEP 4: TIME REFERENCE CALCULATION

[If applicable, convert relative time references]

- Original reference: [e.g., “last year” from May 2022] - Calculated actual time: [e.g., “2021”]

STEP 5: CONTRADICTION CHECK

[If multiple memories contain different information]

- Conflicting information: [describe] - Resolution: [explain which is most recent/reliable]

STEP 6: DETAIL VERIFICATION CHECKLIST

- [] All person names included: [list them] - [] All locations included: [list them] - [] All numbers exact: [list them] - [] All frequencies specific: [list them] - [] All dates/times precise: [list them] - [] All proper nouns preserved: [list them]

STEP 7: ANSWER FORMULATION

[Explain how you’re combining the information to answer the question]

FINAL ANSWER:

[Provide the concise answer with ALL specific details preserved]

—

{context}

Question: {question}

Now, follow the Chain-of-Thought process above to answer the question:

B.2 Judge Prompt

Your task is to label an answer to a question as ‘CORRECT’ or ‘WRONG’. You will be given the following data: (1) a question (posed by one user to another user), (2) a ‘gold’ (ground truth) answer, (3) a generated answer which you will score as CORRECT/WRONG.

The point of the question is to ask about something one user should know about the other user based on their prior conversations. The gold answer will usually be a concise and short answer that includes the referenced topic, for example:

Question: Do you remember what I got the last time I went to Hawaii? Gold answer: A shell necklace

The generated answer might be much longer, but you should be generous with your grading – as long as it touches on the same topic as the gold answer, it should be counted as CORRECT.

For time related questions, the gold answer will be a specific date, month, year, etc. The generated answer might be much longer or use relative time references (like “last Tuesday” or “next month”), but you should be generous with your grading – as long as it refers to the same date or time period as the gold answer, it should be counted as CORRECT. Even if the format differs (e.g., “May 7th” vs “7 May”), consider it CORRECT if it’s the same date.

First, provide a short (one sentence) explanation of your reasoning, then finish with CORRECT or WRONG. Do NOT include both CORRECT and WRONG in your response, or it will break the evaluation script.

1699
1700
1701

Return your response in JSON format with two
keys: "reasoning" for your explanation and "label"
for CORRECT or WRONG.